Final Project Requirements

Requirements for the **final exam project** of the "ASP.NET MVC" course from the official "Applied Programmer" curriculum.

Requirements that are presented below are **mandatory** and **should be completed**. The **demo project**, which was developed during the exercises, **covers them fully**. So, if you have followed **all of the exercise steps thoroughly**, your **project would complete the requirements**.

Make sure **all requirements are satisfied** before the end date for presenting your work.

1. Project Topic

You should **build your own project**, not the **"House Renting System" app**. These are **some ideas** about what you can build:

- · Car rental system
- · Job board system
- · Project tracker (like Trello or Jira)
- · Social network (like Twitter)
- · Online shop

You are free to choose your **own unique topic**. The **user interface** is also of your choice.

2. General Requirements

Your Web application should use the following technologies, frameworks, and development techniques:

- · Use **Visual Studio 2022** or newer
- · The application must be implemented using **ASP.NET Core 8.0** or **ASP.NET Core 10.0** framework.
  - The application must have at least **10** web pages (views)
  - The application must have at least **4 entity models**
  - The application must have at least **4 controllers** (with at least **one API controller**)
- · Adapt the default **ASP.NET Core site template** or get another free theme
  - Use responsive design based on **Twitter Bootstrap** / **Google Material design**
  - Or just design your own
- · Use **Microsoft SQL Server** as database
- · Use **Entity Framework Core** to access your database

    - You should have **database migrations**
    - The database should have **seeded data**
    - You should perform **all CRUD operations** on your **entities**
- · Implement **error handling** and **data validation** to avoid crashes when invalid data is entered
    - Both **client-side** and **server-side**, even at the database(s)
    - Use the **Razor** view engine for generating the UI
    - **Navigation bar** should have **links for accessing pages**, depending on the **user** (logged-in, not logged-in, Admin)
    - Optionally, use **JavaScript** for the **front-end**
    - Optionally, Implement **paging**, **sorting** and **searching** in a page
- · Implement **custom error pages** for "400 Bad Request", "401 Unauthorized" and other error codes
- · Use **AJAX** request to asynchronously load and display data somewhere in your application (by sending a request to the **Web API**)
- · Your **whole business logic** should be in **services**, used by controllers and views
    - **Prevent** from **security vulnerabilities** like **SQL Injection**, **XSS**, **CSRF**, parameter tampering, etc.
- · Use the standard **ASP.NET Identity System** for managing **Users** and **Roles**
    - You should **add custom properties for the user**
    - You should have an **Admin** in a **special Administrator role** and **special access to pages and functionalities**
- · Your **project solution** should be separated to **multiple projects**
- · Display **TempData messages** after some succesful operations
- · Write **unit tests** for your business logic (in a separate project)
    - Implement them with **NUnit**
    - **Mock** external dependencies
    - You should have a **code coverage** of at least **70%**
- · Use **dependency injection**
    - The built-in one in ASP.NET Core is perfectly fine
- · BONUS: **deploy your app** to **Replit** or **Azure**

3. Additional Requirements

Your project **MUST** have a well-structured **architecture** and a well-configured **control flow**.

- · Follow the best practices for **object-oriented design** and **high-quality code** for the Web application:
    - Use the **OOP principles** properly: data encapsulation, inheritance, abstraction and polymorphism
    - Use **exception handling** properly
    - Follow the principles of **strong cohesion** and **loose coupling**

- Correctly **format and structure your code**, name your identifiers and make the code readable

· Make the **user interface (UI) good-looking** and easy to use

- If you provide a broken design, your functionality points will be sanctioned

4. Source Control

Use **GitHub** for a **source control system**

· Your **source code repository** should be **public** from the beginning

- It should contain a **description of your project** in a **README.md file**

· You should have **commits** in at least **7 DIFFERENT** days

· You should have at least **25 commits**

· You should **not commit** any changes after **23:59** on the ***(final date)***

IMPORTANT: the **Source Control Requirements** are **absolutely mandatory**.

5. Project Defense

Each student will have to deliver a **defense** of their work in front of the teacher. They should:

· **Demonstrate** how the application works (very shortly)

· Show the **source code** and explain how it works

· **Answer questions** related to the project (and best practices in general)

Be **well prepared** for presenting maximum of your work for minimum time. **Open the project assets beforehand** to save time.

6. Assessment Criteria

Functionality – **0…20**
Implementing controllers correctly (controllers should do only their work) **– 0...15**
Implementing views correctly (using display and editor templates) **– 0…10**
Implementing services correctly (business logic should be in services, used by controllers and views) **– 0…10**
Unit tests (unit test for some of the controllers using mocking) **– 0…10**
Data validation (validation in the models and input models) **– 0…10**
Additional requirements (inversion of control, mapping, custom error pages, security, etc.) **– 0…15**
Code quality (well-structured code, following the MVC pattern, following SOLID principles, etc.) – **0…10**
Bonus points – 0…5

Контакти:
school@softuni.bg
+359 893 740 775
www.buditel.bg